

프로필 화면 구현 가이드

⚠ 먼저 — 기존 API 교체가 필요합니다

예전에 쓰던 `GET /api/users/me` 는 더 이상 없습니다. 같은 기능이 `GET /api/users/profile` 로 바뀌었습니다.

이미 `/api/users/me` 를 연동했다면 그 주소를 `/api/users/profile` 로 바꿔주세요. 응답에 프로필 사진(`profileImage`) 필드가 새로 추가됐고, 나머지 필드와 토큰 사용 방식은 그대로입니다.

디자인(프로필 화면)을 기준으로, 각 부분을 어떤 API로 구현하는지 화면 순서대로 정리했습니다. 아래 예시 코드는 이 프로젝트의 기존 방식(`src/shared/api/*` 모듈 · `client` · `ApiResponse<T>`)에 맞춰 작성했습니다. (서버 주소는 `VITE_API_BASE_URL`)

먼저 아래 타입과 API 모듈을 만들어 두면 각 화면에서 가져다 씁니다. (기존 `auth.ts` · `todo.ts` 와 같은 패턴)

예시 코드 — 공통 타입

```
// src/shared/types/user.ts
export interface UserProfile {
  userId: number
  email: string
  nickname: string
  role: string
  provider: 'LOCAL' | 'KAKAO' | 'GOOGLE' | 'NAVER'
  profileImage: string | null
}
export interface PresignedUrlData {
  presignedUrl: string
  s3Key: string
}
```

예시 코드 — API 모듈 (src/shared/api/user.ts)

```
// src/shared/api/user.ts (auth.ts·todo.ts와 동일한 방식)
import axios from 'axios'
import client from './client'
import type { ApiResponse } from '@shared/types/auth'
import type { UserProfile, PresignedUrlData } from '@shared/types/user'

// A. 내 프로필 조회
export async function getProfile(
  accessToken: string,
): Promise<ApiResponse<UserProfile>> {
  const res = await client.get<ApiResponse<UserProfile>>('/api/users/profile', {
    headers: { Authorization: `Bearer ${accessToken}` },
  })
}
```

```

    })
    return res.data
  }

// B. 프로필 수정 (이름/이미지 - 보낸 값만 변경)
export async function updateProfile(
  accessToken: string,
  body: { nickname?: string; profileImageKey?: string },
): Promise<ApiResponse<UserProfile>> {
  const res = await client.patch<ApiResponse<UserProfile>>('/api/users/profile', body, {
    headers: { Authorization: `Bearer ${accessToken}` },
  })
  return res.data
}

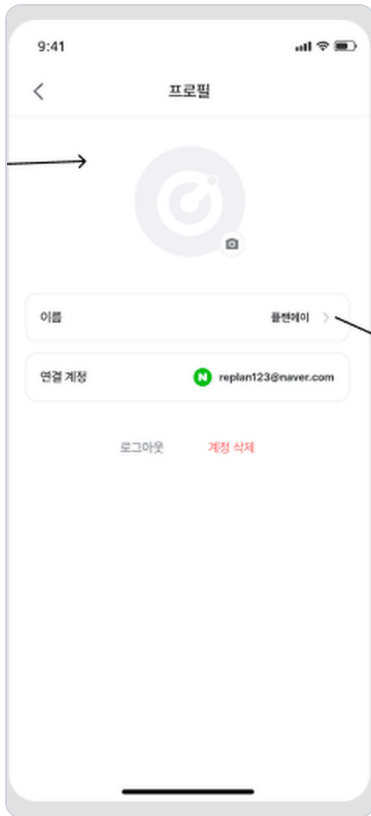
// C. 프로필 이미지 업로드 주소 발급
export async function getProfileImagePresignedUrl(
  accessToken: string,
  filename: string,
  contentType: string,
): Promise<ApiResponse<PresignedUrlData>> {
  const res = await client.get<ApiResponse<PresignedUrlData>>('/api/users/profile/image/presigned-url', {
    {
      params: { filename, contentType },
      headers: { Authorization: `Bearer ${accessToken}` },
    },
  ),
  )
  return res.data
}

// D. 로그아웃
export async function logout(accessToken: string): Promise<ApiResponse<null>> {
  const res = await client.post<ApiResponse<null>>('/api/auth/logout', {}, {
    headers: { Authorization: `Bearer ${accessToken}` },
  })
  return res.data
}

// E. 계정 삭제
export async function deleteAccount(accessToken: string): Promise<ApiResponse<null>> {
  const res = await client.delete<ApiResponse<null>>('/api/users', {
    headers: { Authorization: `Bearer ${accessToken}` },
  })
  return res.data
}

```

A. 프로필 화면 — 첫 진입



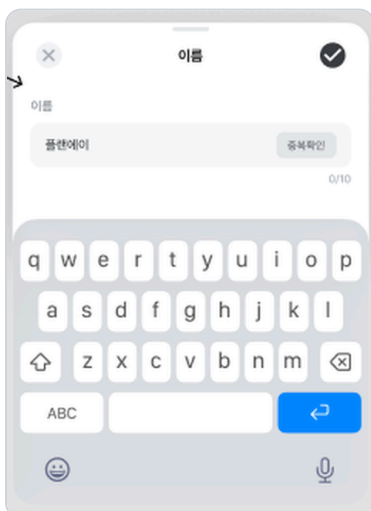
화면을 열 때 **GET /api/users/profile** 를 한 번 호출하고, 응답을 화면에 표시합니다.

화면	응답 필드
프로필 사진	<code>profileImage</code> (없으면 <code>null</code> → 기본 이미지)
이름	<code>nickname</code>
연결 계정 (네이버 아이콘 + 이메일)	<code>provider</code> (아이콘) + <code>email</code>
<code>provider</code> 값에 따라 아이콘 분기: <code>LOCAL/KAKAO/GOOGLE/NAVER</code>	

예시 코드 — 프로필 조회/표시

```
const token = localStorage.getItem('accessToken')!
const res = await getProfile(token)
if (res.success && res.data) {
  const { nickname, profileImage, provider, email } = res.data
  // nickname → 이름, profileImage(null이면 기본 이미지) → 사진
  // provider("NAVER") → 연결 계정 아이콘, email → 계정 주소
}
```

B. 이름 수정 (이름 옆 > → 편집 화면)



① 중복확인 — 이미 있는 `checkNickname()`

(`src/shared/api/auth.ts`) 그대로 사용. `data.available` 가 `false` 면 이미 쓰는 이름입니다.

② 글자수 0/10 — 프론트에서 10자로 제한합니다. (서버는 빈 값만 막고 길이는 검사하지 않습니다.)

③ 저장(체크 버튼) — **PATCH /api/users/profile** 에 `{ nickname }` 전달

예시 코드 — 이름 중복확인 + 저장

```
import { checkNickname } from '@shared/api/auth' // 이미 있는 함수 그대로 사용
import { updateProfile } from '@shared/api/user'

// ① 중복확인
const check = await checkNickname(name)
if (check.success && check.data && !check.data.available) {
  // "이미 사용 중인 이름" 표시
}

// ③ 저장 (체크 버튼)
const res = await updateProfile(token, { nickname: name })
if (!res.success) {
  // res.error.code: 'INVALID_INPUT'(빈 값) / 'DUPLICATE_NICKNAME'(중복)
}
```

C. 프로필 사진 변경 (사진의 카메라 아이콘)

사진은 서버를 거치지 않고 **S3로 직접 업로드**합니다. ① 업로드 주소 받기 → ② S3로 PUT 업로드 → ③ 받은 s3Key 를 프로필 수정으로 전달, 3단계입니다.

예시 코드 — 프로필 사진 업로드 3단계

```
import axios from 'axios'
import { getProfileImagePresignedUrl, updateProfile } from '@shared/api/user'

// 1) 업로드 주소 받기
const r1 = await getProfileImagePresignedUrl(token, file.name, file.type)
if (!r1.success || !r1.data) return
const { presignedUrl, s3Key } = r1.data

// 2) S3로 직접 업로드 - client가 아니라 raw axios, 토큰 넣지 않음
await axios.put(presignedUrl, file, { headers: { 'Content-Type': file.type } })

// 3) 프로필에 반영
await updateProfile(token, { profileImageKey: s3Key })
```

이미지 형식은 jpeg / png / webp / gif 만 가능합니다. (그 외에는 400)

D. 로그아웃 (바텀시트 → 로그아웃)



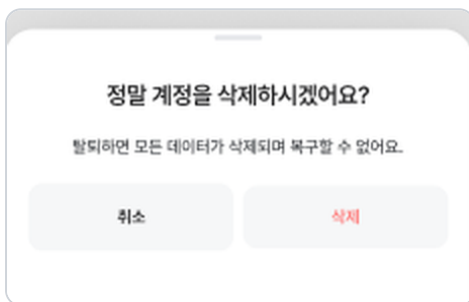
POST /api/auth/logout 호출 후 저장된 토큰·로그인 상태를 비우고 로그인 화면으로 이동시킵니다.

예시 코드 — 로그아웃

```
import { logout } from '@shared/api/user'

await logout(token)
localStorage.clear()
window.location.href = '/' // 로그인 화면
```

E. 계정 삭제 (바텀시트 → 삭제)



DELETE /api/users 호출. (주소가 `/api/users/profile` 이 아니라 `/api/users`) 성공하면 토큰·로그인 상태를 비우고 로그인 화면으로 이동시킵니다.

예시 코드 — 계정 삭제

```
import { deleteAccount } from '@shared/api/user'

await deleteAccount(token)
localStorage.clear()
window.location.href = '/' // 로그인 화면
```

탈퇴하면 개인정보와 본인이 만든 데이터(투두·목표·루틴·태그)가 삭제되고 복구할 수 없습니다. 시트 문구·실제 서버 동작과 일치합니다.

모든 응답은 { `status`, `success`, `data`, `error` } 형태입니다. `success` 가 false면 `error.code` 로 분기하고, 화면 문구는 `error.message` 를 사용합니다. 더 자세한 내용은 Swagger(`/swagger-ui.html`) 참고.